End-to-End Low-Dimensional Autonomous Car For Competition

Igor B. Palhano

***Abstract*—**

***Index Terms*—Autonomous car, Convolutional Networks, Real-time**

I. Introduction

II. Dataset collection

To enable autonomous driving, the authors chose to use an artificial inteligence (AI) system. In this way, the dataset presented in this paper was collected using the car described in **??** on the NXPCUP road. This section describes the acquisition method and the labeling. The referred dataset is available in INSERT LINK.

A. Data Acquisition

The NXPCUP standard road have 55 cm of length, with two black lines (in each board) of 2 cm, furthermore, all the curves of the road have an angle of 45° degrees, and the road can be straightforward, a left curve, a right curve, a zig-zag or a crossroad, as listed in [1]. The collections were carried out in different track formats: 17 were produced from the example track suggested in [1], and 5 were produced on another smaller track. For the first 17, both the direction of the car's turns and the illumination of the road were varied to increase the diversity of the sample obtained during data collection. In all samples, objects were inserted into the camera, and route variations were imposed, thereby increasing the dataset's variability. Each image generated from the videos is an RGB image with dimensions $(360\times1280\times3)$. Furthermore, because it is a real-time application, no image processing is applied to the collected dataset; thus, the dataset consists directly of images acquired by the car's camera. The videos of road laps were recorded using a manual controller, and then split into a batch of images at 10 frame per second (FPS). In the videos, $CHECK_NEW_VALUE$ samples of the road are recorded, with an unequal distribution of road patterns.

B. Data Labeling

Once the dataset had been collected and then split, each image was manually consulted and associated with a string-integer tuple containing the action the car should take and the angle to apply to the steering system. Thus, for straight tracks, intersections, and zigzags, the string "Straight" and the value 0° is assigned; for right turns, the string "Right Curve" and the value 30° is assigned; and for left turns, the string "Left Curve" and the value $-30°$ is assigned. The angle values follow the conventions used in related work, such as the DeepPiCar project in [2].

Increase the dataset description, that is the most import contribution in this paper.

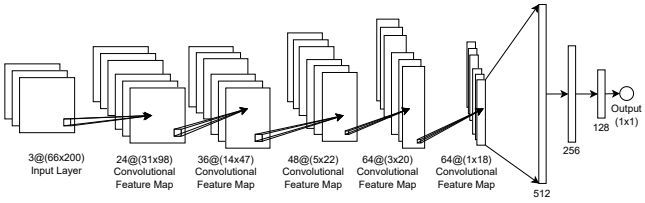


Fig. 1: CNN architecture. The network has 886, 037 parameters and 885, 140 connections.

III. Proposed Neural Network Model

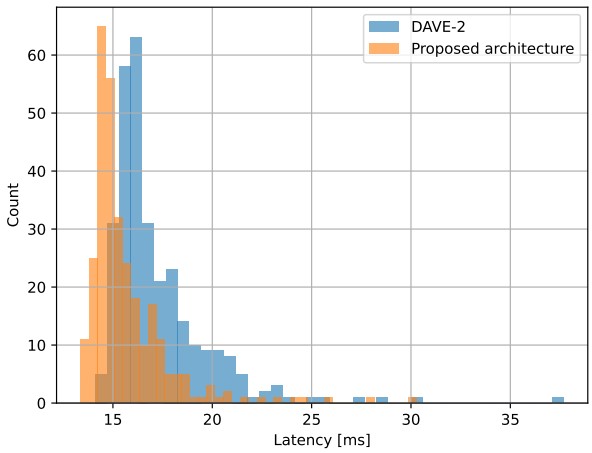
In this section the neural network architecture proposed and the training details for the autonomous car control system are presented.

A. Network Architecture

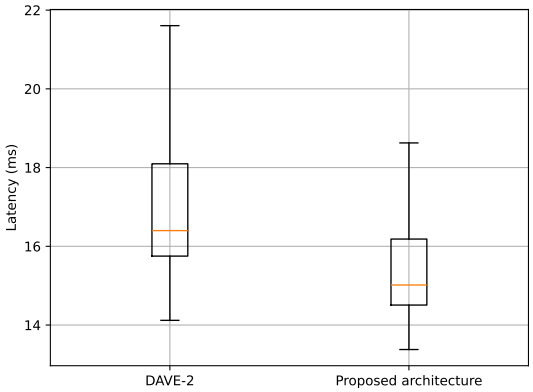
The neural network chosen is an convolutional neural network (CNN), mainly because it processes spatially correlated objects and preserves information. The proposed model is directly inspired in Nvidia DAVE-2, presented in [3]. The network consists of 8 layers: the first 5 are convolutional, and the remaining 3 are fully connected. We use, as in Nvidia DAVE-2, stride layers 2×2 with a kernel of 5×5 for the first 3 convolution layers and a non-stride convolution with a kernel of 3×3 for the last 2 layers; however, the difference is that we use smaller fully-connected layers, with 512, 256, and 128 neurons, respectively. Furthermore, the network is trained to reduce the mean absolute error (MAE) between the given steering angle in the dataset and the one predicted by the neural network.

B. Training Details

After dividing the video frames as presented in section II, the image data was normalized, and augmentation was applied using image shifts and small rotations to avoid compromising the angle of the curves. Variations in image brightness were also applied.



(a) Latencies histogram for DAVE-2 and the proposed architecture.



(b) Latencies box-plot for DAVE-2 and the proposed architecture.

Fig. 2: Latency analysis comparison between DAVE-2 and the proposed architecture.

IV. System Validation

The proposed model and the DAVE-2 network were trained and tested on a machine with an Intel i5-8265U (1.6 GHz) processor and 8 GB of RAM. The training results of both networks on the database presented in section II showed similar performance, with an F1-score of 0.95, indicating a good choice of architecture for the proposed activity.

After training, 300 random samples were generated to verify the latency of the proposed network in comparison to the DAVE-2 network, thus obtaining the data in Fig. 2, where in the left, in orange, we can see the histogram of the proposed network architecture, with a mean latency value of 15.67 ms and a standard deviation of 2.12 ms, and, in blue, we can see the latency histogram of the DAVE-2 network, with an latency mean value of 17.26 ms and a standard deviation of 2.6 ms. Furthermore, on the right, the mean values and their standard deviations for both analyzed models are shown. The proposed network achieved an average latency 10.16% higher than the benchmark. In extreme cases, as for the 99-th percentile, the proportions are still conserved. Those latency measurements show that both models operate within the same order of magnitude, which is possible to observe in Fig. 2(b). In this way, it is possible to conclude that reducing model size does not introduce additional inference overhead, thereby preserving real-time applicability. Nevertheless, for the proposed model, the FLoating-point Operations Per Seconds (FLOPs) count is around 55.13 MFLOPs, and for the DAVE-2 model, around 56.52 MFLOPs, representing 2.5%, which indicates that both models exhibit comparable computational complexity; however, the proposed model has a slight advantage compared to DAVE-2. Moreover, despite the similar computational cost, the proposed model significantly reduces memory requirements due by its reduced size. The DAVE-2 model has $1,580,475$ parameters, and the proposed model has $886,037$ parameters, representing approximately 44% parameter reduction, achieved by using 416 neurons and one fewer dense layer than the DAVE-2 model, which also reflects in a proportional decrease in model size on disk. This reduction is particularly relevant in embedded contexts because memory access affects energy consumption. The direct power measurement was not conducted; however, the model's substantial reduction in energy consumption makes it especially energy-efficient on memory-constrained platforms. Table I summarizes all analyzed information.

Since the convolutional feature extractor remains the same in the DAVE-2 model, the proposed modifications primarily affect the dense layers. In this way, the number of parameters and model size are significantly reduced, while the computational complexity in terms of FLOPs and inference latency remains almost unchanged. Although the reduction in FLOPs is limited, the substantial decrease in parameter count reduces memory access operations, which are well-known in the literature to be a major contributor to energy consumption during inference.

V. Conclusion

This paper proposes a low-dimensional car architecture for competition with an intelligent steering system. The intelligent system was trained using a database produced by the author based on the competition track, and a neural network model based on state-of-the-art autonomous steering was also trained. The model proposed in this work achieves a significant reduction in scale compared to the DAVE-2 model, suggesting it has lower average energy consumption and requires less flash memory, making it preferable for projects where storage limitations are critical. For future work, in order to reduce

Network	DAVE-2	Proposed solution
Parameters	$1,580,475$	$886,037$
FLOPs	56515077	55126617
F1-Score	0.95	0.95
Mean latency	17.2673 ms	15.6744 ms
Latency Std	2.60 ms	2.12 ms
percentile 99 FPS	36 FPS	40 FPS

Table I: Performance data for DAVE-2 and the proposed model.

the number of FLOPs in the network, it would be interesting to propose a new dynamic convolutional layer, thus directly impacting the computational complexity of the model, as well as comparing the proposed model with specialized models, such as architectures based on reinforcement learning for steering angle regression.

ACKNOWLEDGEMENTS

The author would like to thanks the École nationale supérieure d'ingénieurs de Caen - ENSICAEN, for sharing their computational and hardware resources for the development of this work.

References

[1] NXP Semiconductors, *The NXP Cup Rules 2025/2026: The NXP Cup Official Rules, Season 2025/2026, Version 1.0*, NXP Semiconductors, July 2025, version 1.0. [Online]. Available: https://nxpcup.nxp.com/docs/en/supporting-information/NXP-Cup-2026-Rules.pdf

[2] M. G. Bechtel, E. Mcellhiney, M. Kim, and H. Yun, "Deeppicar: A low-cost deep neural network-based autonomous car," in *2018 IEEE 24th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2018, pp. 11–21.

[3] M. Bojarski, D. Del Testa, D. Dworakowski, B. Firner, B. Flepp, P. Goyal, L. D. Jackel, M. Monfort, U. Muller, J. Zhang, X. Zhang, J. Zhao, and K. Zieba, "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016. [Online]. Available: https://arxiv.org/abs/1604.07316